



杭州电子科技大学
《编译原理课程实践》
实验报告

题 目： 4.1 SysY 语言的语法
 和语义规范理解

学 院： 计算机学院

专 业： 计算机科学与技术

班 级： 22052311

学 号： 22050106

姓 名： 王佳琪

完成日期： 2024/12/11

一、实验目的

本实验旨在通过分析 SysY 语言的语法和语义规范，加深对编译原理中语法分析和语义分析部分的理解。通过编写测试程序和分析其语法成分，学生将能够更好地理解语言规范的实现和应用。

二、实验内容与实验要求

1. 阅读和分析 SysY 语言文法

- 仔细阅读文法：深入研究 2022 版的 SysY 语言定义，特别是其文法规则。
- 分析语法成分：对文法中每条规则所定义的语法成分进行详细分析，包括但不限于：
 - 作用：该语法成分在语言中的功能和目的。
 - 限定条件：使用该语法成分时需要满足的条件。
 - 组合情况：该语法成分如何与其他成分组合，形成有效的语句或表达式。
 - 可能产生的句子：给出符合该语法成分的示例句子。

2. 编写测试程序

- 测试程序数量：编写 5-10 个测试程序，确保覆盖 SysY 语言的所有语法规则和常见组合。
- 测试程序复杂度：测试程序应包括简单和复杂的示例，以展示不同语法规则的应用。
- 学号输出要求：测试程序首行应输出队伍学生的学号或者队伍编码，可以通过定义一个变量并赋值来实现。
- 语法规则覆盖：在报告中详细说明每个测试程序覆盖的主要语法规则。

3. 测试程序的编译和执行

- 兼容性：所编写的测试程序应至少能够通过 gcc 等 C 编译器编译执行，以验证其符合 C 语言的子集规则。
- 文件格式：所有测试代码应保存为 .sy 文件。

三、设计方案与算法描述

1. 阅读和分析 SysY 语言文法

1.1 文法规则概述

SysY 是 C 的一个子集，保留了 C 的基本语法和功能，比如变量声明、表达式、函数调用和控制流语句。它的文法规则包括以下主要部分：

- 程序结构：定义程序的入口和全局结构。
- 变量声明：描述如何声明和初始化变量。
- 表达式：定义数学运算和逻辑运算的形式。
- 控制流语句：包括 if-else、while 等语句。
- 函数定义与调用：描述如何定义和调用函数。

1.2 文法成分详细分析

语法成分	作用	限定条件	组合情况
CompUnit	定义程序的整体结构，可以包含全局变量声明和函数定义	至少包含一个 'FuncDef'；可以嵌套多个 'Decl' 和 'FuncDef'	由多个 'Decl' 和 'FuncDef' 组成，可以是递归形式

Decl	声明常量或变量，分配内存或初始化值	类型必须为 `int` 或 `float`	包括 `ConstDecl` 和 `VarDecl`
ConstDecl	声明不可变的常量，可以初始化为常量值或数组	使用 `const` 开头，必须提供初始化值	支持多个常量定义，使用逗号分隔
BType	定义变量或常量的基本数据类型。	必须是 `int` 或 `float`。	与变量、常量、形参等一起使用，作为声明的一部分。
ConstDef	定义一个常量，并为其赋初值，常量不可更改。	必须使用 `const` 关键字进行声明。初值必须是常量表达式（`ConstExp`）。	支持标量常量和数组常量的声明及初始化。
ConstInitVal	表示常量的初始化值，可以是单个常量表达式或初始化数组。	标量常量必须用 `ConstExp` 初始化。数组常量使用 `{}` 进行初始化，元素之间用逗号分隔。	作为常量定义（`ConstDef`）的一部分。
VarDecl	声明变量并分配内存，可以初始化	变量名不能重复，类型必须为 `int` 或 `float`	支持多个变量声明或初始化
VarDef	定义变量，可以为变量初始化。	变量类型必须是 `BType`（`int` 或 `float`）。数组变量必须指定维度，并可选初始化。	支持标量变量、数组变量的声明与初始化。
InitVal	表示变量的初始化值，可以是表达式或初始化数组。	标量变量可以用 `Exp` 赋值。数组变量使用 `{}` 进行初始化，元素之间用逗号分隔。	作为变量定义（`VarDef`）的一部分。
FuncDef	定义一个函数	函数必须有返回类型和函数名，`main` 函数返回 `int`	可与其他函数相互调用，通过 `return` 返回结果
FuncFParams	定义函数的参数列表。	参数的类型必须为 `int` 或 `float`，可以是标量或数组。	可由多个形参通过逗号分隔组成。
FuncFParam	表示单个形参，可以是标量或数组形式。	参数类型必须为 `int` 或 `float`。数组参数可以包含维度信息。	单独或与其他形参一起组成形参列表。
Block	定义一组语句的作用域	使用大括号包裹，可以嵌套块	包含多个 `BlockItem`
BlockItem	表示语句块内的基本组成部分，可以是声	必须是合法的声明或语句。	单独存在或作为语句块的多个子项。

	明或普通语句。		
Stmnt	定义程序的执行逻辑，包括赋值、条件、循环等	条件和表达式必须符合语法规则	可以是简单语句、复杂控制结构或嵌套块
Exp	表达式用于计算值	操作数和操作符必须类型匹配，不能出现未声明的变量	与赋值语句、控制语句等组合形成更复杂的语句
Cond	用于 if 或 while 等条件控制结构，结果必须为布尔值。	只能包含布尔表达式，返回 int 类型且非零为真。	由逻辑与、逻辑或、相等性、关系表达式组合。
LVal	表示可被赋值的变量或数组元素。	必须是已声明的变量或数组，数组访问需提供合法下标。	可以是简单变量或多维数组访问。
PrimaryExp	表示一个基本单元，例如数值、变量或括号的表达式。	数值必须是 IntConst 或 floatConst，变量必须声明。	可嵌套为更复杂的表达式。
Number	表示整型常量（`IntConst`）或浮点型常量（`floatConst`）。	`IntConst` 必须是合法的整数。 `floatConst` 必须是合法的浮点数。	可以在表达式或初始化时使用。
UnaryExp	用于表示带单目运算符的表达式。	运算符为 +、- 或 !；! 仅在条件表达式中使用。	与 PrimaryExp 或递归的一元表达式组合。
UnaryOp	表示对表达式进行单目运算，运算符包括 `+`、`-` 和 `!`。	`!` 仅能出现在条件表达式中。 运算对象必须是合法的表达式或数值。	与表达式（`UnaryExp`）结合使用，进行单目运算。
FuncRParams	表示函数调用时传入的参数列表。	参数必须是合法的表达式（`Exp`）。 参数个数和函数定义中的形参个数必须匹配。	多个参数之间使用逗号分隔。
MulExp	表示乘法、除法或取模运算。	运算符为 */ 或 %，且两端操作数为 int 或 float。	递归定义，支持与一元表达式组合。
AddExp	表示加法或减法运算。	运算两端必须为数值或表达式。	递归定义，包含加法或减法符号。
RelExp	用于比较大小，结果为布尔值。	运算符为 <、>、<= 或 >=，操作数必须为数值。	可递归嵌套，形成更复杂的关系运算。

EqExp	判断两个表达式是否相等或不等，返回布尔值。	运算符为 == 或 !=，操作数必须类型匹配。	可与关系表达式组合。
LAndExp	表示逻辑与运算。	两端必须为布尔值，使用 && 连接。	可递归嵌套形成多层逻辑表达式。
LOrExp	表示逻辑或运算。	两端必须为布尔值，使用 连接。	与逻辑与表达式嵌套组合形成复杂条件。
ConstExp	表示一个可以在编译时计算结果的表达式，通常用于常量初始化或数组维度。	表达式中只能包含常量和合法的运算符。使用的标识符必须是常量。	在常量定义和数组声明中使用。

四、 测试结果

Test1:

```
C:\Users\Gya\Desktop>gcc -x c -S test1.sy -o test1.s
```

```
C:\Users\Gya\Desktop>gcc -x c test1.sy -o test1.out
```

```
C:\Users\Gya\Desktop>test1.out
```

```
wjq: 22050106
```

```
h jy: 22050201
```

```
l rh: 22050229
```

```
t wx: 22050239
```

```
.file "test1.sy"
.text
.def __main; .scl 2; .type 32; .endef
.section .rdata,"dr"
.LC0:
. . . .ascii "wjq: %d\\12\\0"
.LC1:
. . . .ascii "h jy: %d\\12\\0"
.LC2:
. . . .ascii "l rh: %d\\12\\0"
.LC3:
. . . .ascii "t wx: %d\\12\\0"
.text
.globl main
.def main; .scl 2; .type 32; .endef
.seh_proc main
main:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rsp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $22050106, -4(%rbp)
movl $22050201, -8(%rbp)
movl $22050229, -12(%rbp)
movl $22050239, -16(%rbp)

movl -4(%rbp), %eax
movl %eax, %edx
leaq .LC0(%rip), %rcx
call printf
movl -8(%rbp), %eax
movl %eax, %edx
leaq .LC1(%rip), %rcx
call printf
movl -12(%rbp), %eax
movl %eax, %edx
leaq .LC2(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rsp
popq %rbp
ret
.seh_endproc
.ident "GCC: (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0"
.def printf; .scl 2; .type 32; .endef
```

Test2:

```
C:\Users\Gya\Desktop>gcc -x c -S test2.sy -o test2.s

C:\Users\Gya\Desktop>gcc -x c test2.sy -o test2.out

C:\Users\Gya\Desktop>test2.out
1.0 2.0 3.0 4.0
```

```
.file "test2.sy"
.text
.def __main; .scl 2; .type 32; .endef
.section .rdata,"dr"
.LC4:
.asciz "%.1f \0"
.text
.globl main
.def main; .scl 2; .type 32; .endef
.seh_proc main
main:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $64, %rsp
.seh_stackalloc 64
.seh_endprologue
call __main
movss .LC0(%rip), %xmm0
movss %xmm0, -32(%rbp)
movss .LC1(%rip), %xmm0
movss %xmm0, -28(%rbp)
movss .LC2(%rip), %xmm0
movss %xmm0, -24(%rbp)
movss .LC3(%rip), %xmm0
movss %xmm0, -20(%rbp)
movl $0, -4(%rbp)
jmp .L2
.L5:
movl $0, -8(%rbp)
jmp .L3
.L4:
movl -8(%rbp), %eax
cltq
movl -4(%rbp), %edx
movslq %edx, %rdx
addq %rdx, %rdx

addq %rdx, %rax
movss -32(%rbp, %rax, 4), %xmm0
cvtss2sd %xmm0, %xmm0
movq %xmm0, %rax
movq %rax, %rdx
movq %rdx, %xmm1
movq %rax, %rdx
leaq .LC4(%rip), %rcx
call printf
addl $1, -8(%rbp)
.L3:
cmpl $1, -8(%rbp)
jle .L4
addl $1, -4(%rbp)
.L2:
cmpl $1, -4(%rbp)
jle .L5
movl $0, %eax
addq $64, %rsp
popq %rbp
ret
.seh_endproc
.section .rdata,"dr"
.align 4
.LC0:
.long 1065353216
.align 4
.LC1:
.long 1073741824
.align 4
.LC2:
.long 1077936128
.align 4
.LC3:
.long 1082130432
.ident "GCC: (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0"
.def printf; .scl 2; .type 32; .endef
```

Test3:

```
C:\Users\Gya\Desktop>gcc -x c -S test3.sy -o test3.s

C:\Users\Gya\Desktop>gcc -x c test3.sy -o test3.out

C:\Users\Gya\Desktop>test3.out
7
```

```
.file "test3.sy"
.text
.globl add
.def add; .scl 2; .type 32; .endef
.seh_proc add
add:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
.seh_stackalloc 48
.seh_endprologue
call __main
movl %ecx, 16(%rbp)
movl %edx, 24(%rbp)
movl 16(%rbp), %edx
movl 24(%rbp), %eax
addl %edx, %eax
popq %rbp
ret
.seh_endproc
.def __main; .scl 2; .type 32; .endef
.section .rdata,"dr"
.LC0:
.asciz "%d\0"
.text
.globl main
.def main; .scl 2; .type 32; .endef
.seh_proc main
main:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rsp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $4, %edx
movl $3, %ecx
call add
movl %eax, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %edx
leaq .LC0(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rsp
popq %rbp
ret
.seh_endproc
.ident "GCC: (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0"
.def printf; .scl 2; .type 32; .endef
```

Test4:

```

C:\Users\Gya\Desktop>gcc -x c -S test4.sy -o test4.s

C:\Users\Gya\Desktop>gcc -x c test4.sy -o test4.out

C:\Users\Gya\Desktop>test4.out
true

```

```

.file    "test4.sy"
.text
.def     __main; .scl 2; .type 32; .endef
.section .rdata,"dr"
.LC0:
.ascii  "true\0"
.text
.globl  main
.def     main; .scl 2; .type 32; .endef
.seh_proc  main
main:
    pushq   %rbp
    .seh_pushreg   %rbp
    movq    %rsp, %rbp
    .seh_setframe %rbp, 0
    subq    $48, %rsp
    .seh_stackalloc 48
    .seh_endprologue
    call    __main
    movl    $5, -4(%rbp)
    movl    $10, -8(%rbp)
    movl    -4(%rbp), %eax
    cmpl    -8(%rbp), %eax
    jge     .L2
    cmpl    $0, -8(%rbp)
    jg      .L3
.L2:
    cmpl    $0, -4(%rbp)
    je      .L4
.L3:
    leaq    .LC0(%rip), %rcx
    call    printf
.L4:
    movl    $0, %eax
    addq    $48, %rsp
    popq    %rbp
    ret
.seh_endproc
.ident    "GCC: (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0"
.def     printf; .scl 2; .type 32; .endef

```

Test5:

```

C:\Users\Gya\Desktop>gcc -x c -S test5.sy -o test5.s

C:\Users\Gya\Desktop>gcc -x c test5.sy -o test5.out

C:\Users\Gya\Desktop>test5.out
23
3.3
1

```

```

.file      "test5.sy"
.text
.def      __main; .scl 2; .type 32; .endef
.section .rdata,"dr"
.LC1:
.ascii    "%d\\12%.1f\\12%d\\0"
.text
.globl    main
.def      main; .scl 2; .type 32; .endef
.seh_proc main
main:
    pushq   %rbp
    .seh_pushreg   %rbp
    movq    %rsp, %rbp
    .seh_setframe %rbp, 0
    subq    $48, %rsp
    .seh_stackalloc 48
    .seh_endprologue
    call    __main
    movl    $23, -4(%rbp)
    movss   .LC0(%rip), %xmm0
    movss   %xmm0, -8(%rbp)
    movl    $1, -12(%rbp)
    cvtss2sd    -8(%rbp), %xmm0
    movl    -12(%rbp), %edx
    movq    %xmm0, %rax
    movq    %rax, %rcx
    movq    %rax, %xmm0
    movl    -4(%rbp), %eax
    movl    %edx, %r9d
    movq    %rcx, %xmm2
    movq    %xmm0, %r8
    movl    %eax, %edx
    leaq    .LC1(%rip), %rcx
    call    printf
    movl    $0, %eax
    addq    $48, %rsp
    popq    %rbp
    ret
    .seh_endproc
    .section .rdata,"dr"
    .align 4
.LC0:
    .long   1079334229
    .ident  "GCC: (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0"
    .def     printf; .scl 2; .type 32; .endef

```

五、 源代码

// Test 1: 学号输出

// 涉及语法规则: CompUnit, FuncDef, Decl, Stmt

#include <stdio.h>

```

int main() {
    int student_id1 = 22050106;
    int student_id2 = 22050201;
    int student_id3 = 22050229;
    int student_id4 = 22050239;
    printf("wjq: %d\\n", student_id1);
    printf("hgy: %d\\n", student_id2);
    printf("lrh: %d\\n", student_id3);
    printf("twx: %d\\n", student_id4);
    return 0;
}

```

// Test 2: 变量声明与赋值、循环

// 涉及语法规则: Program, FuncDef, VarDecl, InitVal, Stmt, Exp, LVal, PrimaryExp, Number

#include<stdio.h>

```

int main(){
    float matrix[2][2] = {{1.0, 2.0}, {3.0, 4.0}};
    for( int i=0; i<2; i++){
        for( int j=0; j<2; j++){
            printf("%.1f",matrix[i][j]);
        }
    }
    return 0;
}

```

```
}
```

// Test 3: 函数定义和调用

// 涉及语法规则: FuncDef, FuncType, FuncFParams, FuncFParam, Block, Stmt, Exp, AddExp。

```
#include<stdio.h>
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int main() {  
    int result = add(3, 4);  
    printf("%d",result);  
    return 0;  
}
```

// Test 4: 条件语句, 逻辑表达式

// 涉及语法规则: Stmt, Cond, RelExp, Block, Cond, LOrExp, LAndExp, EqExp, AddExp, Exp。

```
#include<stdio.h>
```

```
int main() {  
    int a=5;  
    int b=10;  
    if (a < b && b > 0 || a != 0) {  
        printf("true");  
    }  
    return 0;  
}
```

// Test 5: 运算表达式

// 涉及语法规则: Exp, AddExp, MulExp, UnaryExp, PrimaryExp, Number。

```
#include<stdio.h>
```

```
int main() {  
    int sum = 3 + 4 * 5;  
    float quotient = 10.0 / 3.0;  
    int remainder = 10 % 3;  
    printf("%d\n%.1f\n%d",sum,quotient,remainder);  
    return 0;  
}
```